

Giới thiệu NumPy

Mảng, vector hóa, broadcasting và tính toán số

Duc-Minh Vu

SLSCM Lab – FDA – NEU

Ngày 10 tháng 8 năm 2026

Lộ trình bài học

- 1 NumPy là gì
- 2 Vì sao NumPy hữu ích
- 3 Cài đặt và import
- 4 Tạo mảng
- 5 Thuộc tính mảng
- 6 Indexing và slicing
- 7 Reshaping, resizing, stacking, splitting
- 8 Broadcasting
- 9 Số học và tổng hợp
- 10 Universal functions
- 11 Toán tử Boolean
- 12 Linear algebra
- 13 Số ngẫu nhiên và thống kê
- 14 Tích hợp và lỗi thường gặp

Mục tiêu học tập

Sau bài học này, người học có thể:

- ▶ Giải thích vai trò của NumPy trong tính toán số.
- ▶ Phân biệt mảng NumPy với list Python.
- ▶ Tạo mảng một chiều và mảng nhiều chiều.
- ▶ Kiểm tra shape, size, ndim và dtype.
- ▶ Sử dụng indexing, slicing, reshaping, stacking và splitting.
- ▶ Áp dụng số học vector hóa, broadcasting, tổng hợp và universal functions.
- ▶ Thực hiện đại số tuyến tính cơ bản và thống kê mô tả.
- ▶ Sinh giá trị ngẫu nhiên và hiểu tính tái lập kết quả.
- ▶ Giải thích cách NumPy kết nối với Pandas, SciPy và Scikit-learn.

Điều kiện tiên quyết và cài đặt

Kiến thức nền khuyến nghị

- ▶ Biến, list, vòng lặp và hàm cơ bản trong Python.
- ▶ Một môi trường Python có thể chạy được: Jupyter Lưu ýbook, JupyterLab, Google Colab hoặc tương tự.

Cài đặt và import

```
pip install numpy
```

```
import numpy as np  
print(np.__version__)
```

Cách đọc các câu lệnh NumPy

Hầu hết câu lệnh NumPy có cùng một mẫu:

```
np.function(input, option1=value1, option2=value2)
```

- ▶ `np`: bí danh chuẩn của thư viện NumPy.
- ▶ `function`: phép toán ta muốn NumPy thực hiện.
- ▶ `input`: mảng, list, shape hoặc giá trị số được truyền vào hàm.
- ▶ `option=value`: đối số có tên dùng để điều khiển hành vi của câu lệnh.

Ví dụ

`np.arange(0, 10, 2)` tạo các giá trị bắt đầu từ 0, dừng trước 10, với bước nhảy 2.

Mẫu câu lệnh: đối tượng, thuộc tính, phương thức, hàm

Dạng	Ý nghĩa
<code>np.array([1,2,3])</code>	Gọi một hàm NumPy để tạo mảng.
<code>arr.shape</code>	Đọc một thuộc tính mô tả mảng.
<code>arr.reshape(2,3)</code>	Gọi một phương thức gắn với đối tượng mảng hiện có.
<code>np.mean(arr)</code>	Gọi một hàm NumPy để tóm tắt mảng.
<code>arr.mean()</code>	Phiên bản gọi theo kiểu phương thức, tương đương với nhiều phép toán phổ biến.

Teaching note

Sinh viên nên xác định ba thành phần trong mỗi câu lệnh: đầu vào, phép toán và đầu ra.

Ví dụ ngắn: giải thích từng dòng

```
import numpy as np

scores = np.array([7, 8, 6, 9])
bonus = scores + 1
average = scores.mean()
print(bonus)
print(average)
```

- ▶ `import numpy as np`: nạp NumPy và đặt tên ngắn là `np`.
- ▶ `np.array([...])`: chuyển list Python thành mảng NumPy.
- ▶ `scores + 1`: cộng 1 vào mọi phần tử bằng vector hóa.
- ▶ `scores.mean()`: tính giá trị trung bình của mảng.
- ▶ `print(...)`: hiển thị kết quả.

NumPy là gì?

NumPy, viết tắt của **Numerical Python**, là thư viện Python cốt lõi cho tính toán số nhanh.

Đối tượng trung tâm của nó là **N-dimensional array**, called `ndarray`. Một mảng NumPy có thể biểu diễn:

- ▶ Một vectơ một chiều.
- ▶ Một ma trận hai chiều.
- ▶ Một tensor ba chiều.
- ▶ Các cấu trúc số có số chiều cao hơn.

Vì sao quan trọng

NumPy lưu trữ dữ liệu đồng nhất hiệu quả và hỗ trợ phép toán nhanh trên toàn bộ mảng.

NumPy cung cấp gì?

- ▶ Các phép toán mảng nhanh.
- ▶ Tính toán vector hóa.
- ▶ Broadcasting.
- ▶ Các hàm đại số tuyến tính.
- ▶ Các hàm thống kê.
- ▶ Sinh số ngẫu nhiên.
- ▶ Tích hợp với Pandas, SciPy, Matplotlib và Scikit-learn.

Các đặc trưng chính

Đặc trưng	Ý nghĩa
ndarray	Mảng nhiều chiều hiệu quả với một kiểu dữ liệu chung.
Vectorization	Áp dụng phép tính lên toàn bộ mảng mà không cần vòng lặp Python tường minh.
Broadcasting	Thực hiện phép toán trên các mảng có shape khác nhau nhưng tương thích.
Linear algebra	Nhân ma trận, định thức, nghịch đảo, trị riêng và tích vectơ.
Statistics	Trung bình, trung vị, phương sai, độ lệch chuẩn, percentile và quantile.
Integration	Làm việc với Pandas, SciPy, Matplotlib, Scikit-learn và Statsmodels.

Kiểm tra nhanh: NumPy là gì?

Câu 1. Cấu trúc dữ liệu chính trong NumPy là gì?

- ▶ A. DataFrame
- ▶ B. ndarray
- ▶ C. dictionary
- ▶ D. tuple

Câu 2. Đúng hay sai? NumPy được thiết kế chủ yếu cho tính toán số.

Tại sao nên học NumPy?

NumPy cung cấp nền tảng hiệu quả cho công việc tính toán số trong Python.

- ▶ Thực hiện phép toán vector hóa nhanh hơn nhiều vòng lặp Python thông thường.
- ▶ Lưu trữ dữ liệu số đồng nhất gọn hơn list Python chuẩn.
- ▶ Cung cấp hàm tối ưu cho ma trận, đại số tuyến tính và biến đổi.
- ▶ Hỗ trợ sinh số ngẫu nhiên và phân tích thống kê.
- ▶ Biểu diễn phép toán toán học phức tạp bằng cú pháp ngắn gọn.
- ▶ Là nền tảng tính toán số cho nhiều thư viện khoa học dữ liệu.

List Python và mảng NumPy

List Python: các đối tượng hỗn hợp

```
values = [10, 2.5, "Python", True]
```

- ▶ Cấu trúc chứa đối tượng linh hoạt.
- ▶ Không tối ưu cho mảng số lớn.

NumPy array: các giá trị đồng nhất

```
import numpy as np  
values = np.array([10, 20, 30, 40])
```

- ▶ Cấu trúc đều đặn.
- ▶ Tính toán số hiệu quả.

Ví dụ: nhân các giá trị

List Python

```
values = [1, 2, 3, 4]
result = []

for value in values:
    result.append(value * 10)

print(result)
# [10, 20, 30, 40]
```

Vector hóa NumPy

```
import numpy as np
values = np.array([1, 2, 3, 4])

result = values * 10
print(result)
# [10 20 30 40]
```

Ý tưởng chính

Cùng một phép toán được áp dụng cho tất cả phần tử cùng lúc.

Kiểm tra nhanh: tại sao học NumPy?

Câu 1. Vì sao mảng NumPy hiệu quả cho tính toán số?

- ▶ A. Chúng luôn chứa văn bản
- ▶ B. Chúng dùng cấu trúc mảng đồng nhất và được tối ưu
- ▶ C. Chúng không dùng bộ nhớ
- ▶ D. Chúng tự động kết nối internet

Câu 2. Biểu thức nào nhân mọi phần tử của mảng NumPy `a` bằng 10?

- ▶ A. `a * 10`
- ▶ B. `a.append(10)`
- ▶ C. `a.add("10")`
- ▶ D. `a.sort(10)`

Tạo mảng NumPy

Mảng NumPy có thể được tạo từ list, tuple, range của Python hoặc các hàm dựng sẵn của NumPy.

```
import numpy as np

a = [9, 3, 3, 5]
arr = np.array(a)
print(arr)
# [9 3 3 5]
```

```
arr = np.array([10, 20, 30, 40])
print(arr)
```

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print(matrix)
```

Mảng nhiều chiều

Mảng ba chiều có thể biểu diễn các ma trận xếp chồng hoặc tensor.

```
tensor = np.array([  
    [  
        [1, 2],  
        [3, 4]  
    ],  
    [  
        [5, 6],  
        [7, 8]  
    ]  
])  
print(tensor)
```

Diễn giải

Các chiều thường tương ứng với quan sát, đặc trưng, thời gian, kênh hoặc kịch bản.

Các hàm tạo mảng thường dùng

```
np.zeros(5)
np.zeros((2, 3))
np.ones((2, 3))
np.full((2, 3), 7)
```

```
np.arange(0, 10, 2)
# [0 2 4 6 8]

np.linspace(0, 1, 5)
# [0.    0.25 0.5   0.75 1.]

np.eye(3)
```

Các lệnh chính: tạo mảng

Lệnh	Ý nghĩa và khi nào dùng
<code>np.array(data)</code>	Chuyển list, tuple hoặc chuỗi lồng nhau thành ndarray.
<code>np.zeros(shape)</code>	Tạo mảng toàn số 0; hữu ích khi khởi tạo.
<code>np.ones(shape)</code>	Tạo mảng toàn số 1; hữu ích cho mask, trọng số hoặc khởi tạo.
<code>np.full(shape, value)</code>	Tạo mảng được lấp đầy bằng một giá trị cho trước.
<code>np.arange(start, stop, step)</code>	Sinh giá trị cách đều; stop không được lấy.
<code>np.linspace(start, stop, n)</code>	Sinh đúng n giá trị cách đều, mặc định bao gồm hai đầu mút.
<code>np.eye(n)</code>	Tạo ma trận đơn vị $n \times n$.

Kiểm tra nhanh: tạo mảng

Câu 1. Hàm nào chuyển list Python thành mảng NumPy?

- ▶ A. `np.array()`
- ▶ B. `np.list()`
- ▶ C. `np.convert()`
- ▶ D. `np.ndarray_list()`

Câu 2. Hàm nào tạo các giá trị cách đều giữa hai đầu mút?

- ▶ A. `np.linspace()`
- ▶ B. `np.stack()`
- ▶ C. `np.split()`
- ▶ D. `np.mean()`

Thuộc tính mảng: quan sát một mảng

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(arr.ndim)      # 2
print(arr.shape)     # (2, 3)
print(arr.size)      # 6
print(arr.dtype)
print(arr.itemsize)
```

How to read these commands

Mỗi câu lệnh yêu cầu NumPy mô tả mảng: số chiều, shape, tổng số phần tử, kiểu dữ liệu và bộ nhớ mỗi phần tử.

Thuộc tính mảng: ý nghĩa từng thuộc tính

Thuộc tính	Ý nghĩa
ndim	Số chiều. Vectơ có 1 chiều; ma trận có 2 chiều.
shape	Kích thước theo từng chiều. Ví dụ, (2, 3) nghĩa là 2 hàng và 3 cột.
size	Tổng số phần tử trong mảng.
dtype	Kiểu dữ liệu lưu trong mảng, chẳng hạn số nguyên hoặc số thực.
itemsize	Số byte dùng cho mỗi phần tử.

Các lệnh chính: kiểm tra thuộc tính mảng

Thuộc tính	Nó cho biết điều gì
<code>arr.ndim</code>	Số trục/số chiều của mảng.
<code>arr.shape</code>	Độ dài của mảng theo từng trục; ví dụ, (3,4) means 3 rows and 4 columns.
<code>arr.size</code>	Tổng số phần tử được lưu trữ.
<code>arr.dtype</code>	Kiểu dùng để lưu mỗi phần tử, chẳng hạn <code>int64</code> or <code>float64</code> .
<code>arr.itemsize</code>	Number of bytes used by one element.
<code>arr.nbytes</code>	Tổng số byte dùng bởi tất cả phần tử mảng.

Thói quen hữu ích

Trước khi reshape, broadcasting hoặc mô hình hóa, hãy kiểm tra shape and dtype; nhiều lỗi NumPy bắt nguồn từ shape hoặc kiểu dữ liệu không như mong đợi.

Kiểm tra nhanh: thuộc tính mảng

Câu 1. Thuộc tính nào trả về kích thước của mảng?

- ▶ A. shape
- ▶ B. mean
- ▶ C. append
- ▶ D. index

Câu 2. Thuộc tính nào trả về tổng số phần tử?

- ▶ A. size
- ▶ B. dtype
- ▶ C. ndim
- ▶ D. itemsize

Indexing mảng

Indexing một chiều

```
arr = np.array([10, 20, 30, 40])

print(arr[0])    # 10
print(arr[2])    # 30
print(arr[-1])   # 40
```

Indexing hai chiều

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix[0, 1]) # 2
print(matrix[1, 2]) # 6
```

Quy ước indexing

Python và NumPy dùng chỉ số bắt đầu từ 0.

Slicing mảng

Slicing một chiều

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4])
# [20 30 40]

print(arr[::-2])
# [10 30 50]
```

Slicing hai chiều

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix[0:2, 1:3])
# [[2 3]
#   [5 6]]
```

View và bản sao

Nhiều phép slicing của NumPy trả về **view** của mảng gốc thay vì một bản sao độc lập.

```
arr = np.array([10, 20, 30, 40])  
  
view = arr[1:3]  
view[0] = 999  
  
print(arr)  
# [ 10 999  30  40]
```

Tạo bản sao độc lập khi cần:

```
copy = arr[1:3].copy()
```

Quan trọng

Thay đổi một view có thể làm thay đổi mảng gốc.

Các lệnh chính: indexing và slicing

Cú pháp	Ý nghĩa
<code>arr[i]</code>	Chọn phần tử ở vị trí <code>i</code> ; chỉ số âm đếm từ cuối mảng.
<code>arr[a:b]</code>	Chọn các vị trí <code>a</code> through <code>b-1</code> .
<code>arr[a:b:s]</code>	Slice with step size <code>s</code> .
<code>matrix[i,j]</code>	Select row <code>i</code> , column <code>j</code> .
<code>matrix[r1:r2,c1:c2]</code>	Trích xuất một mảng con hình chữ nhật.
<code>arr[mask]</code>	Boolean indexing: retain values where <code>mask</code> is <code>True</code> .
<code>arr.copy()</code>	Make independent data when a view should not modify the original.

Kiểm tra nhanh: indexing và slicing

Câu 1. Điều gì diễn ra khi `arr[0]` trả về gì?

- ▶ A. The first element
- ▶ B. The last element
- ▶ C. The array shape
- ▶ D. Kích thước mảng

Câu 2. In a two-dimensional array, `matrix[1, 2]` refers to:

- ▶ A. Row index 1 and column index 2
- ▶ B. Row 1 only
- ▶ C. Column 2 only
- ▶ D. The array dimensions

Câu 3. Đúng hay sai? Một lát cắt NumPy có thể dùng chung bộ nhớ với mảng gốc.

Đổi hình dạng mảng

Reshape thay đổi kích thước/chiều của mảng mà không thay đổi dữ liệu.

```
arr = np.arange(12)
matrix = arr.reshape(3, 4)
print(matrix)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]
```

Dùng -1 để suy ra một chiều:

```
matrix = arr.reshape(2, -1)
```

Làm phẳng mảng:

```
flat = ma.atrien.flatten()
flat_view = ma.atrien.ravel()
```

`flatten()` trả về bản sao; `ravel()` thường trả về view nếu có thể.

Thay đổi kích thước mảng

`resize()` thay đổi shape và có thể thay đổi số phần tử.

```
arr = np.array([1, 2, 3, 4])
resized = np.resize(arr, (2, 3))
print(resized)
# [[1 2 3]
#  [4 1 2]]
```

So sánh

- ▶ `reshape()` yêu cầu tổng số phần tử phải tương thích.
- ▶ `resize()` có thể tạo tổng kích thước khác bằng cách lặp lại giá trị.

Ghép mảng

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
np.vstack((a, b))  
# [[1 2 3]  
#  [4 5 6]]
```

```
np.hstack((a, b))  
# [1 2 3 4 5 6]
```

```
np.stack((a, b), axis=0)
```

Mục đích

Stacking ghép các mảng thành mảng lớn hơn theo một trục được chọn.

Tách mảng

Splitting chia mảng thành các mảng nhỏ hơn.

```
arr = np.array([1, 2, 3, 4, 5, 6])
parts = np.split(arr, 3)
print(parts)
# [array([1, 2]), array([3, 4]), array([5, 6])]
```

Đối với mảng nhiều chiều:

```
matrix = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8]
])

left, right = np.hsplit(matrix, 2)
```

Các lệnh chính: reshape, stack và split

Lệnh	Ý nghĩa
<code>arr.reshape(r,c)</code>	Sắp xếp lại cùng các phần tử vào shape mới tương thích.
<code>arr.reshape(...,-1)</code>	Yêu cầu NumPy tự suy ra một chiều.
<code>arr.flatten()</code>	Trả về dạng phẳng 1-D <i>copy</i> .
<code>arr.ravel()</code>	Trả về view phẳng khi có thể.
<code>np.resize(arr, shape)</code>	Thay đổi tổng kích thước nếu cần; giá trị có thể lặp lại.
<code>np.vstack((a,b))</code>	Ghép mảng theo chiều dọc (hướng hàng).
<code>np.hstack((a,b))</code>	Ghép mảng theo chiều ngang.
<code>np.stack((a,b), axis=k)</code>	Thêm trục mới và ghép theo trục đó.
<code>np.split(arr,n)</code>	Chia mảng thành n phần bằng nhau khi có thể.
<code>np.hsplit(matrix,n)</code>	Chia ma trận theo chiều cột.

Kiểm tra nhanh: reshaping và splitting

Câu 1. Phương thức nào đổi mảng từ shape này sang shape khác?

- ▶ A. `reshape()`
- ▶ B. `mean()`
- ▶ C. `split()`
- ▶ D. `sort_index()`

Câu 2. Điều gì diễn ra khi `-1` nghĩa là gì trong `reshape()`?

- ▶ A. NumPy nên tự suy ra chiều đó
- ▶ B. Xóa một chiều
- ▶ C. Đảo ngược mảng
- ▶ D. Chuyển giá trị thành số âm

Câu 3. Hàm nào ghép mảng theo chiều dọc? `np.vstack()`, `np.mean()`, `np.split()`, or `np.random()`?

Broadcasting: ý tưởng cốt lõi

Broadcasting cho phép NumPy thực hiện phép toán giữa các mảng có shape tương thích.

```
arr = np.array([1, 2, 3, 4])  
result = arr + 10  
print(result)  
# [11 12 13 14]
```

Quy tắc cơ bản

Bắt đầu từ các chiều ngoài cùng bên phải, hai chiều tương thích khi:

- ▶ chúng bằng nhau; hoặc
- ▶ một trong chúng là 1.

Broadcasting giữa các mảng

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

row = np.array([10, 20, 30])
result = matrix + row
print(result)
# [[11 22 33]
#  [14 25 36]]
```

Shape không tương thích

```
a = np.ones((2, 3))
b = np.ones((2, 2))
# a + b gây lỗi broadcasting
```

Các phép toán số học cơ bản

NumPy hỗ trợ số học theo từng phần tử.

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
print(a + b)    # [5 7 9]  
print(a - b)    # [-3 -3 -3]  
print(a * b)    # [4 10 18]  
print(a / b)  
print(a ** 2)   # [1 4 9]  
print(b % a)
```

Phân biệt quan trọng

thực hiện nhân theo từng phần tử, không phải nhân ma trận.

Các lệnh chính: số học và broadcasting

Biểu thức	Ý nghĩa
$a + b, a - b$	Cộng và trừ theo từng phần tử.
$a * b$	Nhân theo từng phần tử.
a / b	Chia theo từng phần tử.
$a ** p$	Nâng mọi phần tử lên lũy thừa p .
$a \% b$	Phần dư theo từng phần tử.
<code>array + scalar</code>	Broadcast số vô hướng đến mọi phần tử.
<code>matrix + row</code>	Broadcast một hàng 1-D tương thích qua các hàng của ma trận.

Kiểm tra shape trước

Broadcasting so sánh các chiều từ phải sang trái. Các chiều phải bằng nhau hoặc một trong chúng phải bằng 1.

Kiểm tra nhanh: broadcasting và số học

Câu 1. Broadcasting cho phép điều gì?

- ▶ A. Phép toán trên các mảng có shape khác nhau nhưng tương thích
- ▶ B. Truyền dữ liệu tự động qua internet
- ▶ C. Chuyển mảng thành tệp
- ▶ D. Loại bỏ mọi chiều

Câu 2. Hai chiều tương thích broadcasting khi chúng bằng nhau hoặc:

- ▶ A. Một trong chúng bằng 1
- ▶ B. Cả hai đều âm
- ▶ C. Cả hai là văn bản
- ▶ D. Tổng của chúng bằng 0

Câu 3. Điều gì diễn ra khi $a * b$ làm gì với các mảng cùng shape?

Các hàm tổng hợp

Các hàm tổng hợp tóm tắt một mảng.

```
arr = np.array([9, 3, 3, 5])

print(arr.sum())      # 20
print(arr.mean())     # 5.0
print(arr.min())
print(arr.max())
print(np.median(arr))
print(np.var(arr))
print(np.std(arr))
```

Cách dùng điển hình

Aggregation biến mảng thô thành thống kê tóm tắt.

Tổng hợp theo trục

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix.sum(axis=0))
# [5 7 9]

print(matrix.sum(axis=1))
# [ 6 15]
```

- ▶ axis=0: tổng hợp theo chiều hàng, tạo kết quả theo cột.
- ▶ axis=1: tổng hợp theo chiều cột, tạo kết quả theo hàng.

Universal functions

Universal function, hay **ufunc**, thực hiện phép toán theo từng phần tử trên mảng.

```
arr = np.array([1, 4, 9, 16])
print(np.sqrt(arr))
# [1.  2.  3.  4.]

print(np.exp(np.array([0, 1, 2])))
```

```
values = np.array([1, np.e, np.e**2])
print(np.log(values))

angles = np.array([0, np.pi / 2, np.pi])
print(np.sin(angles))
```

Các ví dụ khác: `np.abs()`, `np.round()`.

Các lệnh chính: tổng hợp và universal functions

Lệnh	Ý nghĩa
<code>arr.sum(), arr.mean()</code>	Tổng và trung bình số học.
<code>arr.min(), arr.max()</code>	Giá trị nhỏ nhất và lớn nhất.
<code>np.median(arr)</code>	Trung vị.
<code>np.var(arr), np.std(arr)</code>	Phương sai và độ lệch chuẩn.
<code>func(..., axis=0)</code>	Tổng hợp theo chiều hàng, tạo một kết quả cho mỗi cột.
<code>func(..., axis=1)</code>	Tổng hợp theo chiều cột, tạo một kết quả cho mỗi hàng.
<code>np.sqrt(arr)</code>	Căn bậc hai theo từng phần tử.
<code>np.exp(arr), np.log(arr)</code>	Hàm mũ và log tự nhiên theo từng phần tử.
<code>np.sin(arr)</code>	Hàm sin theo từng phần tử; góc tính bằng radian.
<code>np.abs(arr), np.round(arr)</code>	Giá trị tuyệt đối và làm tròn.

Kiểm tra nhanh: tổng hợp và ufunc

Câu 1. Hàm nào tính trung bình số học?

- ▶ A. `np.mean()`
- ▶ B. `np.stack()`
- ▶ C. `np.dot()`
- ▶ D. `np.reshape()`

Câu 2. Trong mảng hai chiều, `axis=0` thường tổng hợp theo chiều nào?

- ▶ A. Theo hàng, tạo kết quả theo cột
- ▶ B. Theo cột, tạo kết quả theo hàng
- ▶ C. Tên tệp
- ▶ D. Kiểu dữ liệu

Câu 3. Hàm nào tính căn bậc hai theo từng phần tử?

So sánh và lọc Boolean

NumPy hỗ trợ so sánh theo từng phần tử.

```
arr = np.array([10, 20, 30, 40])
print(arr > 20)
# [False False  True  True]

selected = arr[arr > 20]
print(selected)
# [30 40]
```

Kết hợp điều kiện:

```
selected = arr[(arr >= 20) & (arr <= 30)]
print(selected)
# [20 30]
```

Dùng & cho AND, | cho OR và ~ cho NOT.

Các lệnh chính: toán tử Boolean

Biểu thức	Ý nghĩa
<code>arr > value</code>	Tạo mảng Boolean từ phép so sánh theo từng phần tử.
<code>arr[arr > value]</code>	Lọc và chỉ trả về các giá trị thỏa điều kiện.
<code>(cond1) & (cond2)</code>	AND logic theo từng phần tử.
<code>(cond1) (cond2)</code>	OR logic theo từng phần tử.
<code>~mask</code>	NOT logic theo từng phần tử.
<code>np.where(cond, a, b)</code>	Chọn a nơi điều kiện đúng và b trong trường hợp ngược lại.

Cú pháp quan trọng

Dùng dấu ngoặc quanh các phép so sánh kết hợp vì `&` and `|` có độ ưu tiên khác với toán tử so sánh.

Kiểm tra nhanh: toán tử Boolean

Câu 1. Điều gì diễn ra khi `arr[arr > 20]` trả về gì?

- ▶ A. Các phần tử lớn hơn 20
- ▶ B. Kích thước mảng
- ▶ C. Kiểu dữ liệu của mảng
- ▶ D. Tất cả phần tử được chuyển thành giá trị Boolean

Thảo luận. Vì sao lọc Boolean hữu ích trong phân tích dữ liệu?

Nhân ma trận

```
A = np.array([
    [1, 2],
    [3, 4]
])

result = np.dot(A, A)
print(result)
# [[ 7 10]
#  [15 22]]

result = A @ A
```

Nhân theo từng phần tử và nhân ma trận

- ▶ `A` `A`: nhân theo từng phần tử.
- ▶ `A @ A`: nhân ma trận.

Các phép toán ma trận

```
print(A.T)

determinant = np.linalg.det(A)
print(determinant)

inverse = np.linalg.inv(A)
print(inverse)
```

```
A = np.array([
    [2, 1],
    [1, 3]
])

b = np.array([8, 13])
x = np.linalg.solve(A, b)
print(x)
```

Điều kiện

Nghịch đảo chỉ tồn tại với ma trận vuông không suy biến.

Trị riêng và tích vectơ

```
eigenvalues, eigenvectors = np.linalg.eig(A
    )

print(eigenvalues)
print(eigenvectors)
```

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.inner(a, b))
print(np.outer(a, b))
print(np.dot(a, b))
print(np.vdot(a, b))
```

Với mảng một chiều thực, dot and vdot thường giống nhau. vdot cũng xử lý liên hợp phức.

Các lệnh chính: đại số tuyến tính

Lệnh	Ý nghĩa
$A @ B$	Nhân ma trận; cú pháp dễ đọc được ưu tiên.
<code>np.dot(A,B)</code>	Tích vô hướng / tích ma trận tùy theo số chiều.
$A.T$	Chuyển vị mảng.
<code>np.linalg.det(A)</code>	Tính định thức của ma trận vuông.
<code>np.linalg.inv(A)</code>	Tính nghịch đảo của ma trận vuông không suy biến.
<code>np.linalg.giải(A,b)</code>	Solve $Ax = b$ trực tiếp; thường tốt hơn việc tính $A^{-1}b$.
<code>np.linalg.eig(A)</code>	Trả về trị riêng và vectơ riêng.
<code>np.inner(a,b)</code>	Tích trong.
<code>np.outer(a,b)</code>	Tích ngoài.
<code>np.vdot(a,b)</code>	Tích vô hướng vectơ có liên hợp phức khi cần.

Kiểm tra nhanh: đại số tuyến tính

Câu 1. Toán tử nào thực hiện nhân ma trận?

- ▶ A. @
- ▶ B. %
- ▶ C. //
- ▶ D. &

Câu 2. Hàm nào tính nghịch đảo ma trận?

- ▶ A. `np.linalg.inv()`
- ▶ B. `np.mean()`
- ▶ C. `np.resize()`
- ▶ D. `np.random()`

Câu 3. Giải thích sự khác nhau giữa $A * B$ and $A @ B$.

Sinh số ngẫu nhiên

Với mã mới, NumPy thường dùng một đối tượng sinh số ngẫu nhiên.

```
rng = np.random.default_rng()
```

```
rng = np.random.default_rng(42)
values = rng.integers(
    low=1,
    high=10,
    size=5
)
print(values)
```

```
values = rng.normal(
    loc=0,
    scale=1,
    size=5
)
print(values)
```

Lưu ý

Bộ sinh số ngẫu nhiên của NumPy dùng cho tính toán số và thống kê; không nên xem là bộ sinh bảo mật mật mã.

Các phân phối thường dùng

```
rng = np.random.default_rng(42)

rng.uniform(low=0, high=1, size=5)
rng.normal(loc=0, scale=1, size=5)
rng.binomial(n=10, p=0.5, size=5)
rng.poisson(lam=3, size=5)
rng.exponential(scale=2, size=5)
rng.chisquare(df=4, size=5)
```

Tính tái lập

Dùng cùng seed giúp tái tạo cùng chuỗi giả ngẫu nhiên trong môi trường tương thích.

Các hàm thống kê

```
data = np.array([12, 15, 18, 20, 25])

print(np.mean(data))
print(np.median(data))
print(np.var(data))
print(np.std(data))
print(np.percentile(data, 25))
print(np.quantile(data, 0.75))
```

Quy ước tổng thể và mẫu

Mặc định, `np.var()` and `np.std()` use `ddof=0`. Với quy ước mẫu thường dùng, hãy dùng `ddof=1`.

Giá trị số bị thiếu

NumPy thường biểu diễn giá trị số bị thiếu bằng `np.nan`.

```
data = np.array([10.0, 20.0, np.nan, 40.0])
print(np.mean(data))
# nan

print(np.nanmean(data))
print(np.nanmedian(data))
print(np.nanstd(data))

print(np.isnan(data))
clean_data = data[~np.isnan(data)]
```

Các quy trình xử lý dữ liệu thiếu đầy đủ hơn thường được thực hiện bằng Pandas.

Các lệnh chính: số ngẫu nhiên và thống kê

Lệnh	Ý nghĩa
<code>np.random.default_rng(seed)</code>	Tạo bộ sinh số ngẫu nhiên hiện đại; seed hỗ trợ tái lập.
<code>rng.integers(low,high,size)</code>	Sinh số nguyên ngẫu nhiên; high không được lấy.
<code>rng.uniform(...)</code>	Lấy mẫu từ phân phối đều.
<code>rng.normal(loc,scale,size)</code>	Lấy mẫu từ phân phối chuẩn với trung bình loc and độ lệch chuẩn scale.
<code>rng.binomial(...)</code> / <code>poisson(...)</code>	Lấy mẫu từ các phân phối rời rạc phổ biến.
<code>np.percentile(data,q)</code>	Trả về percentile q trên thang 0–100.
<code>np.quantile(data,q)</code>	Trả về quantile q trên thang 0–1.
<code>np.std(data,ddof=1)</code>	Quy ước độ lệch chuẩn mẫu.
<code>np.isnan(data)</code>	Xác định NaN giá trị.
<code>np.nanmean(data)</code>	Trung bình khi bỏ qua NaN.

Kiểm tra nhanh: số ngẫu nhiên và thống kê

Câu 1. Phương thức nào tạo bộ sinh số ngẫu nhiên hiện đại của NumPy?

- ▶ A. `np.random.default_rng()`
- ▶ B. `np.random.file()`
- ▶ C. `np.create_random_array()`
- ▶ D. `np.random.text()`

Câu 2. Vì sao seed hữu ích?

- ▶ A. Nó giúp tái tạo chuỗi giả ngẫu nhiên
- ▶ B. Nó làm cho giá trị thực sự không thể dự đoán
- ▶ C. Nó loại bỏ mọi giá trị ngẫu nhiên
- ▶ D. Nó chuyển dữ liệu thành chuỗi

Câu 3. Tham số nào cho quy ước độ lệch chuẩn mẫu thường dùng? `ddof=1`, `axis=-100`, `dtype="sample"`, or `copy=False`?

Các phép toán vector hóa

Vectorization áp dụng phép toán lên toàn bộ mảng cùng lúc.

```
a = np.arange(5)
result = a * 10
print(result)
# [ 0 10 20 30 40]
```

```
a = list(range(5))
result = []
for value in a:
    result.append(value * 10)
print(result)
```

Lợi ích điển hình

Mã ngắn hơn, dễ đọc hơn, nhanh hơn với mảng số lớn và phù hợp hơn với quy trình khoa học.

Logic điều kiện vector hóa

```
arr = np.array([10, 20, 30, 40])

labels = np.where(
    arr >= 30,
    "high",
    "low"
)

print(labels)
# ['low' 'low' 'high' 'high']
```

Diễn giải

Điều kiện vector hóa hữu ích cho gán nhãn, sàng lọc, lọc và biến đổi dữ liệu.

Các lệnh chính: vector hóa

Mẫu lệnh	Ý nghĩa
<code>arr * scalar</code>	Áp dụng phép toán số học cho mọi phần tử mà không cần vòng lặp Python tường minh.
<code>np.sqrt(arr)</code>	Áp dụng ufunc theo từng phần tử trong mã NumPy đã tối ưu.
<code>arr >= threshold</code>	Đánh giá điều kiện cho toàn mảng cùng lúc.
<code>np.where(cond,a,b)</code>	Biến đổi điều kiện theo kiểu vector hóa.

Ý tưởng cốt lõi

Vectorization mô tả *cách ta biểu diễn tính toán*: phép toán được viết cho mảng thay vì lặp thủ công qua từng phần tử.

Kiểm tra nhanh: vector hóa

Câu 1. Ưu điểm chính của vector hóa là gì?

- ▶ A. Nó áp dụng phép toán mảng mà không cần vòng lặp Python tường minh
- ▶ B. Nó chuyển mọi mảng thành văn bản
- ▶ C. Nó loại bỏ nhu cầu dùng bộ nhớ
- ▶ D. Nó ngăn mọi lỗi

Thảo luận. Khi nào mã vector hóa dễ bảo trì hơn mã dựa trên vòng lặp?

Lưu ý về bộ nhớ

Mảng NumPy thường lưu giá trị đồng nhất trong bố cục bộ nhớ đều đặn.

```
arr = np.array([1, 2, 3, 4], dtype=np.int32)
print(arr.nbytes)

small_values = np.array([1, 2, 3, 4], dtype=np.int8)
print(small_values.dtype)
print(small_values.nbytes)
```

Lựa chọn kiểu dữ liệu

Kiểu dữ liệu nhỏ hơn có thể giảm bộ nhớ, nhưng phải biểu diễn được miền giá trị và độ chính xác cần thiết.

Chuyển đổi kiểu dữ liệu

```
arr = np.array([1.2, 2.8, 3.5])  
integers = arr.astype(np.int32)  
print(integers)  
# [1 2 3]
```

Quan trọng

Chuyển số thực sang số nguyên sẽ loại bỏ phần thập phân.

Sắp xếp và tìm kiếm

```
arr = np.array([9, 3, 7, 1])
sorted_arr = np.sort(arr)
print(sorted_arr)
# [1 3 7 9]

arr = np.array([10, 20, 30, 40])
indices = np.where(arr > 20)
print(indices)

arr = np.array([1, 2, 2, 3, 3, 3])
values, counts = np.unique(arr, return_counts=True)
print(values)
print(counts)
```

Làm việc với ảnh

Ảnh số có thể được biểu diễn bằng mảng NumPy.

- ▶ Ảnh xám: mảng hai chiều.
- ▶ Ảnh màu: mảng ba chiều chứa chiều cao, chiều rộng và kênh màu.

```
image = np.array([
    [0, 128, 255],
    [255, 128, 0],
    [50, 100, 150]
])
print(image.shape)
# (3, 3)

brighter = np.clip(image + 30, 0, 255)
```

`np.clip()` giữ các giá trị trong khoảng hợp lệ.

Các lệnh chính: bộ nhớ, kiểu dữ liệu, sắp xếp và tìm kiếm

Lệnh	Ý nghĩa
<code>arr.nbytes</code>	Tổng bộ nhớ dùng bởi phần tử mảng.
<code>arr.astype(dtype)</code>	Chuyển giá trị mảng sang kiểu dữ liệu mới.
<code>np.sort(arr)</code>	Trả về giá trị đã sắp xếp.
<code>np.where(condition)</code>	Trả về chỉ số thỏa điều kiện khi chỉ cung cấp điều kiện.
<code>np.unique(arr)</code>	Trả về giá trị duy nhất.
<code>np.unique(arr, return_counts=True)</code>	Trả về giá trị duy nhất và tần suất.
<code>np.clip(arr,low,high)</code>	Giới hạn mọi giá trị trong khoảng chỉ định; hữu ích cho cường độ ảnh.

Kiểm tra nhanh: bộ nhớ, tìm kiếm, ảnh

Câu 1. Thuộc tính nào trả về số byte dùng bởi phần tử mảng?

- ▶ A. `nbytes`
- ▶ B. `ndim`
- ▶ C. `mean`
- ▶ D. `shape`

Câu 2. Hàm nào trả về các giá trị duy nhất của mảng?

- ▶ A. `np.unique()`
- ▶ B. `np.reshape()`
- ▶ C. `np.random()`
- ▶ D. `np.outer()`

Câu 3. Ảnh xám thường được biểu diễn bằng loại mảng nào?

Tích hợp với Pandas

Một Pandas Series or DataFrame có thể trao đổi dữ liệu với NumPy.

```
import numpy as np
import pandas as pd

arr = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

df = pd.DataFrame(arr, columns=["A", "B"])
array_from_df = df.to_numpy()

df["A_sqrt"] = np.sqrt(df["A"])
```

Tích hợp với Scikit-learn

Nhiều mô hình Scikit-learn chấp nhận mảng NumPy làm đầu vào.

```
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 6, 8])

model = LinearRegression()
model.fit(X, y)

prediction = model.predict(np.array([[5]]))
print(prediction)
```

NumPy hỗ trợ ma trận đặc trưng, vectơ mục tiêu, dự đoán mô hình, tiền xử lý số và phép tính đánh giá.

Các lỗi NumPy thường gặp

Shape không khớp

```
a = np.ones((2, 3))  
b = np.ones((2, 2))  
# a + b raises an error
```

Reshape không hợp lệ

```
arr = np.arange(10)  
# arr.reshape(3, 4) raises an error
```

Chia cho 0

```
arr = np.array([1.0, 0.0])  
print(1 / arr)
```

Tràn số nguyên

```
arr = np.array([127], dtype=np.int8)  
print(arr + 1)
```

Thực hành tốt

- ▶ Import NumPy bằng `import numpy as np`.
- ▶ Dùng phép toán vector hóa thay cho vòng lặp Python khi phù hợp.
- ▶ Check `shape`, `ndim`, and `dtype` trước các phép toán phức tạp.
- ▶ Chỉ dùng broadcasting khi hành vi `shape` mong muốn đã rõ.
- ▶ Phân biệt nhân theo phần tử với nhân ma trận.
- ▶ Dùng `.copy()` khi cần mảng độc lập.
- ▶ Chọn kiểu dữ liệu cẩn thận để cân bằng miền giá trị, độ chính xác và bộ nhớ.
- ▶ Dùng seed cố định khi tính tái lập quan trọng.
- ▶ Treat NaN, vô cực và tràn số một cách tường minh.

Các lệnh chính: tích hợp với công cụ khoa học dữ liệu

Lệnh	Vai trò
<code>pd.DataFrame(arr,...)</code>	Tạo DataFrame Pandas từ mảng NumPy.
<code>df.to_numpy()</code>	Trích xuất giá trị DataFrame thành mảng NumPy.
<code>np.sqrt(df["A"])</code>	Áp dụng ufunc NumPy cho dữ liệu Pandas tương thích.
<code>model.fit(X,y)</code>	Huấn luyện mô hình Scikit-learn bằng dữ liệu đặc trưng và mục tiêu dạng mảng.
<code>model.predict(Xnew)</code>	Tạo dự đoán cho quan sát mới dạng mảng.

Mối liên hệ

Mảng NumPy là định dạng trao đổi dữ liệu số phổ biến giữa Pandas, SciPy, Matplotlib và Scikit-learn.

Kiểm tra nhanh: tích hợp và lỗi

Câu 1. Phương thức Pandas nào chuyển DataFrame thành mảng NumPy?

- ▶ A. `to_numpy()`
- ▶ B. `to_list_only()`
- ▶ C. `as_matrix_text()`
- ▶ D. `convert_numpy_file()`

Câu 2. Trong học máy, mảng NumPy hai chiều thường biểu diễn:

- ▶ A. Ma trận đặc trưng
- ▶ B. Tên tệp
- ▶ C. Tiêu đề biểu đồ
- ▶ D. Trình cài đặt package

Câu 3. Vì sao `np.arange(10).reshape(3, 4)` bị lỗi?

Tóm tắt nội dung I

Chủ đề	Ý chính
NumPy	Thư viện Python cốt lõi cho tính toán số.
ndarray	Mảng đồng nhất N chiều.
Vectorization	Phép toán mảng không cần vòng lặp Python tường minh.
Broadcasting	Phép toán trên các shape khác nhau nhưng tương thích.
Indexing và slicing	Truy cập và trích xuất phần tử mảng.
Reshaping	Thay đổi số chiều/kích thước mảng.

Tóm tắt nội dung II

Chủ đề	Ý chính
Aggregation	Tổng, trung bình, nhỏ nhất, lớn nhất, phương sai và độ lệch chuẩn.
Universal functions	Các hàm toán học nhanh theo từng phần tử.
Linear algebra	Nhân ma trận, nghịch đảo, định thức và trị riêng.
Random generation	Sinh giá trị từ các phân phối xác suất.
Statistics	Trung bình, trung vị, phương sai, độ lệch chuẩn và percentile.
Integration	Làm việc chặt chẽ với Pandas, SciPy và Scikit-learn.

Thực hành: tạo mảng

Bài tập 1

- 1 Tạo mảng chứa các giá trị từ 1 đến 20.
- 2 Reshape nó thành 4×5 ma trận.
- 3 In shape, số chiều, size và kiểu dữ liệu.

Bài tập 2

Sử dụng 4×5 matrix:

- 1 Trích xuất hàng đầu tiên.
- 2 Trích xuất cột cuối cùng.
- 3 Trích xuất phần trung tâm 2×3 .
- 4 Chọn tất cả giá trị chẵn bằng lọc Boolean.

Thực hành: thống kê và broadcasting

Bài tập 3: Statistics

Tạo mảng gồm mười giá trị số và tính trung bình, trung vị, phương sai, độ lệch chuẩn mẫu, nhỏ nhất, lớn nhất và các percentile 25, 50, 75.

Bài tập 4: Broadcasting

- 1 Tạo một ma trận 3×4 .
- 2 Tạo một mảng một chiều với bốn giá trị.
- 3 Cộng mảng một chiều vào từng hàng của ma trận.
- 4 Giải thích vì sao broadcasting hợp lệ.

Thực hành: đại số tuyến tính và giá trị thiếu

Bài tập 5: Linear algebra

Given

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 8 \\ 13 \end{bmatrix},$$

tính định thức của A , nghịch đảo của A , giải $Ax = b$, và kiểm tra bằng $A @ x$.

Bài tập 6: Missing values

Given `[10.0, np.nan, 20.0, 30.0, np.nan, 40.0]`, đếm giá trị thiếu, tính trung bình khi bỏ qua giá trị thiếu, loại bỏ giá trị thiếu và thay thế giá trị thiếu bằng trung vị.

Câu hỏi suy ngẫm cuối bài

- ▶ Chủ đề NumPy nào quan trọng nhất cho quy trình phân tích dữ liệu?
- ▶ Khi nào nên kiểm tra `shape` trước khi chạy phép toán?
- ▶ Vectorization và broadcasting giảm vòng lặp thủ công như thế nào?
- ▶ Vì sao cần phân biệt view và bản sao?
- ▶ Khi nào nên dùng `ddof=1` thay cho mặc định?
- ▶ Lỗi NumPy thường gặp nào bạn dễ gặp nhất trong thực hành?